Project Executable Files and Run Instructions

Phase 7: Project Documentation

1. Repository Contents

The supplied source archive contains separate frontend and backend directories. The frontend uses React, TypeScript and Vite. The backend uses Python and FastAPI and includes the API routes, AI service, schemas, Supabase client setup and dependency list. The source archive does not include private deployment credentials and should not be expected to contain application screenshots or a recorded demonstration video.

2. Frontend Setup

1. Install a supported Node.js release and npm.
2. Extract the project archive and open a terminal in the frontend directory.
3. Install the listed dependencies using npm install.
4. Start the development server using npm run dev. On Windows PowerShell, npm.cmd run dev can be used if npm.ps1 is blocked by execution policy.
5. Open the local URL printed by Vite and verify that the page renders.

3. Backend Setup

1. Install Python 3 and create a virtual environment in the backend directory.
2. Activate the virtual environment.
3. Install dependencies from requirements.txt.
4. Set the required Groq and Supabase environment variables privately.
5. Run the FastAPI service with uvicorn from the backend directory.
6. Open /health and record the response before testing itinerary generation.

4. Environment Configuration

The backend requires provider credentials for Groq and a Supabase connection. The exact environment variable names should be read from the source code and deployment settings. Do not copy real values into a public README, report, screenshot or Git commit. For local development, use a private .env file excluded from version control.

5. Deployed Resources

- GitHub repository: https://github.com/Arpit0027-lab/trip-genius-ai
- Frontend: https://trip-genius-ai-nine.vercel.app
- Backend: https://trip-genius-ai.onrender.com
- Health endpoint: https://trip-genius-ai.onrender.com/health

6. Troubleshooting

Frontend cannot reach the backend

Confirm the configured API base URL, check browser developer tools for network errors, and verify that the backend is running. CORS settings should allow the deployed frontend origin.

Itinerary generation fails

Check the backend logs for model quota errors, invalid credentials, JSON parsing failures or timeouts. Never expose the API key when sharing the log.

Trip does not save

Verify Supabase connection settings, table names, schema and permissions. Check the backend response and database logs before repeating destructive actions.

7. Submission Package Notes

Keep the source archive, report, phase documents, actual app screenshots and demo video as separate clearly named deliverables. The report should include links to the deployed application but should not contain secrets. Mark local setup as verified only after another person can follow the steps successfully.